

## Contents

|                                                 |          |
|-------------------------------------------------|----------|
| <b>1 Setup</b>                                  | <b>2</b> |
| <b>2 Preparing the dataset</b>                  | <b>2</b> |
| <b>3 Inspecting the dataset</b>                 | <b>2</b> |
| <b>4 Training a kNN model</b>                   | <b>4</b> |
| 4.1 Prepare the train and test data . . . . .   | 4        |
| 4.2 Train the model . . . . .                   | 4        |
| 4.3 Test the model . . . . .                    | 5        |
| 4.4 Extra: predicting custom outcomes . . . . . | 6        |
| 4.5 Extra: visualising the model . . . . .      | 6        |
| 4.6 Extra: LOOCV . . . . .                      | 7        |
| 4.7 Extra: random seeds . . . . .               | 7        |
| 4.8 Extra: choice of tuneLength . . . . .       | 8        |

## 1 Setup

This lab focuses on the `flights` dataset from the `nycflights13` package.

```
library(dplyr)
library(caret)
library(ggplot2)

data("flights", package="nycflights13")
```

We are focused on the air times and gain of delay of long distance flights by carrier UA.

## 2 Preparing the dataset

Following the criteria set out in the lab sheet, we prepare the dataset.

```
df <- flights %>% filter(air_time > 550, carrier == "UA") %>%
  mutate(gain=arr_delay - dep_delay) %>%
  select(air_time, gain)
```

## 3 Inspecting the dataset

Look at the structure and summary data of the dataset using `str()` and `summary()`.

```
str(df)
```

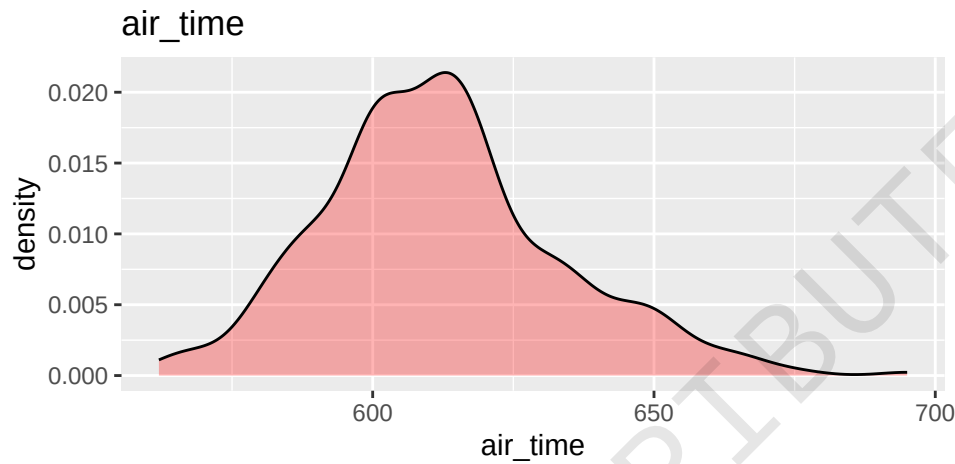
```
## tibble [359 x 2] (S3: tbl_df/tbl/data.frame)
## $ air_time: num [1:359] 656 634 628 623 600 610 620 645 667 624 ...
## $ gain    : num [1:359] 21 -4 -6 -5 -39 -26 -9 13 45 -1 ...
```

```
summary(df)
```

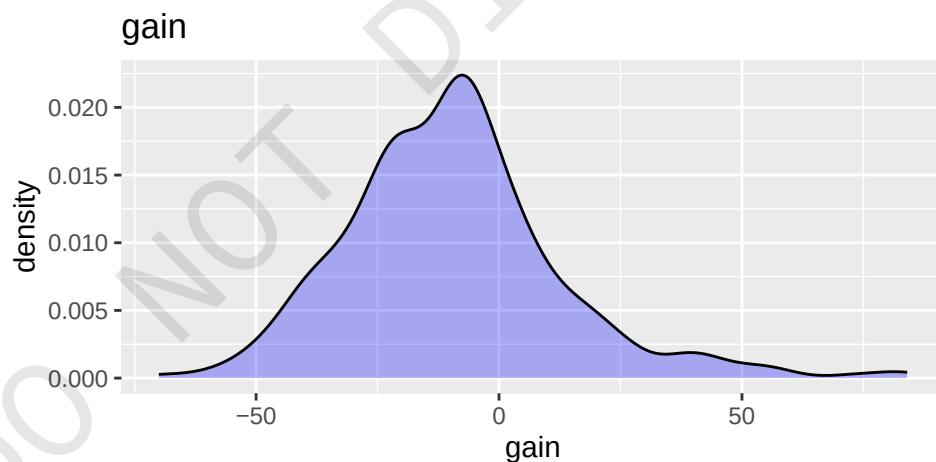
```
##      air_time      gain
##  Min.   :562.0   Min.   : -70.000
##  1st Qu.:599.0   1st Qu.: -23.000
##  Median :611.0   Median : -10.000
##  Mean   :612.1   Mean    :  -9.599
##  3rd Qu.:622.5   3rd Qu.:   1.000
##  Max.   :695.0   Max.    :  84.000
```

You may also choose to visualise the distribution of each variable independently.

```
ggplot(df) + geom_density(aes(x=air_time), fill="red", alpha=0.3) +  
  ↳ labs(title="air_time")
```



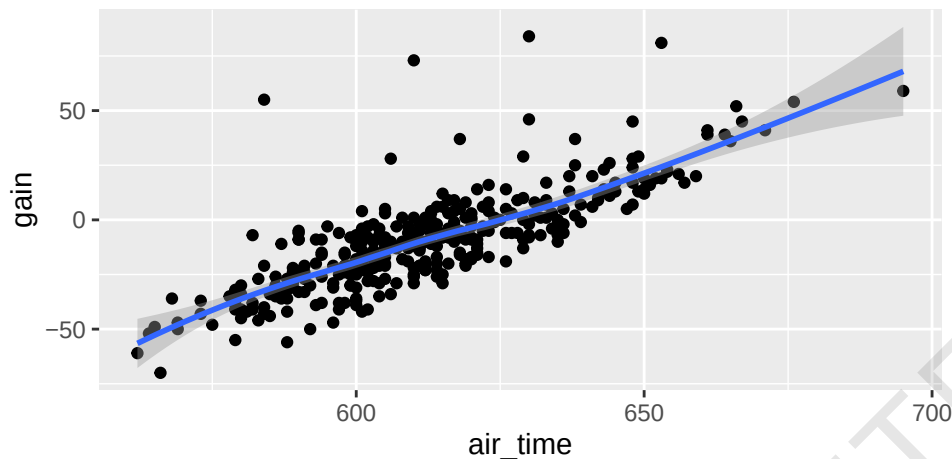
```
ggplot(df) + geom_density(aes(x=gain), fill="blue", alpha=0.3) +  
  ↳ labs(title="gain")
```



This is the same dataset used, matching the graph.

```
ggplot(df, aes(air_time, gain)) + geom_point() + geom_smooth()
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



## 4 Training a kNN model

### 4.1 Prepare the train and test data

```
set.seed(100)
train.idx <- sample(nrow(df), 0.8*nrow(df))
# using the same seed, you should get the same indices.
head(train.idx)
```

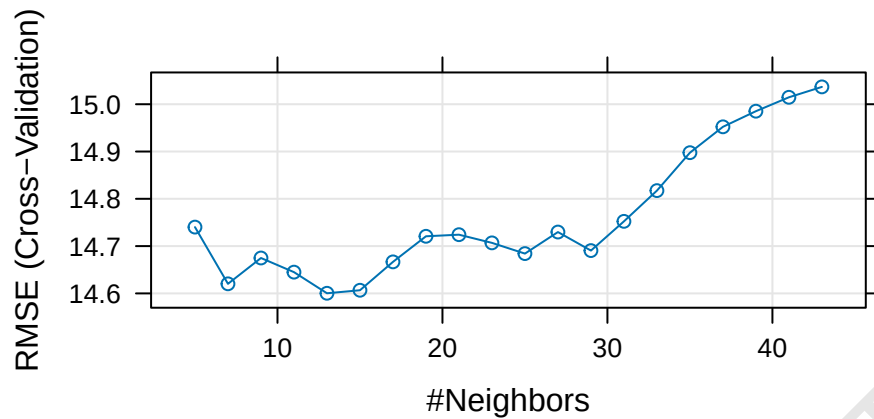
```
## [1] 202 358 112 206 4 311
```

```
train.data <- df[train.idx,]
test.data <- df[-train.idx,]
```

### 4.2 Train the model

For this dataset, there are 287 observations in the training dataset. Following the guideline to have 40-50 observations in each set, we set 6 folds in the cross-validation. The best RMSE was achieved with  $k = 13$ .

```
set.seed(101)
knn <- train(gain ~ air_time, data=train.data, method="knn",
             trControl=trainControl("cv", number=6),
             preProcess=c("center", "scale"),
             tuneLength=20)
plot(knn)
```



```
knn$bestTune
```

```
## # A tibble: 1 x 1
##       k
##   <int>
## 1     13
```

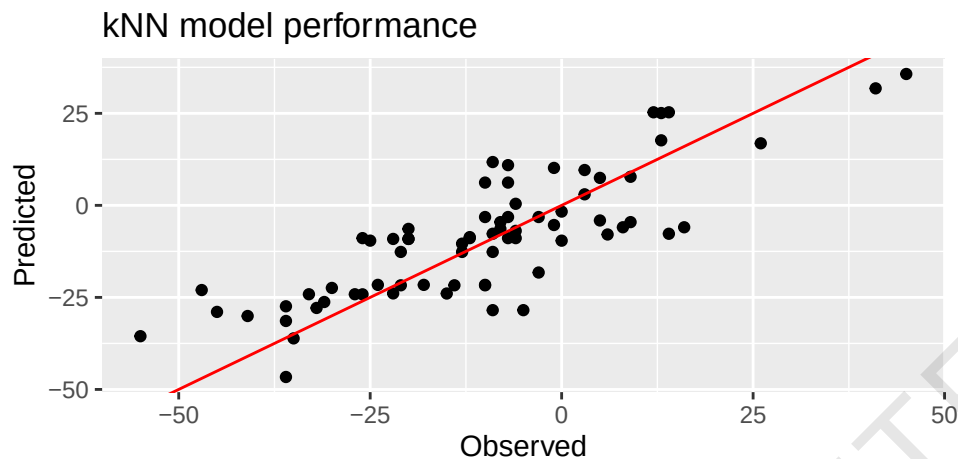
### 4.3 Test the model

RMSE: 10.86391

```
yhat <- predict(knn, test.data)
y <- test.data$gain
RMSE(yhat, y)
```

```
## [1] 10.86391
```

```
ggplot(data.frame(yhat, y), aes(y, yhat)) + geom_point() +
  geom_abline(aes(slope=1, intercept=0), col="red") +
  labs(title="kNN model performance", x="Observed", y="Predicted")
```



#### 4.4 Extra: predicting custom outcomes

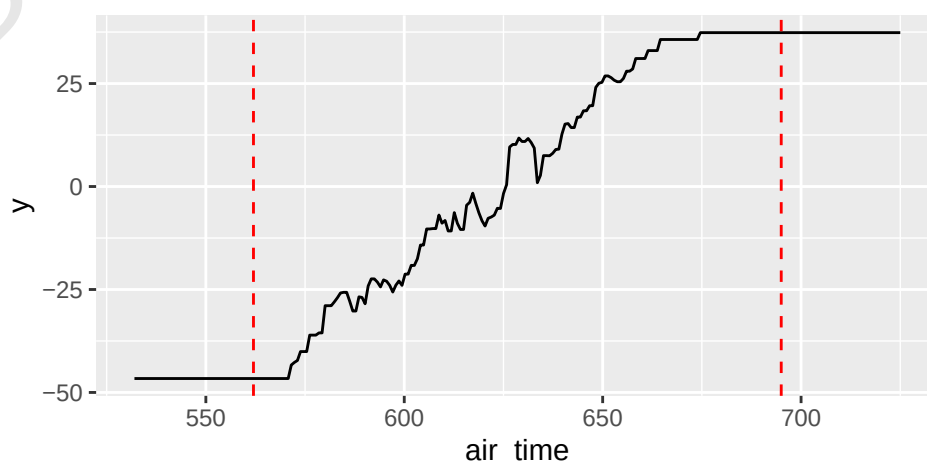
Remember that the training data consists of observations with `air_time > 550`. If we try and predict any new outcomes with `air_time < 550`, it will always have the same result, since the nearest neighbours are always going to be the same.

```
x <- data.frame(air_time=c(2,20,200))
predict(knn, x)
```

#### 4.5 Extra: visualising the model

Since the kNN model is just the training data, we can visualise it. This model has one input and one output, so it is simpler to visualise than the tutorial example. `air_time` ranges from 562 to 695.

```
x <- data.frame(air_time=seq(532, 725, length.out=250))
y <- predict(knn, x)
ggplot(cbind(x, y)) + geom_line(aes(air_time, y)) +
  geom_vline(xintercept=c(562, 695), linetype="dashed", color="red")
```



#### 4.6 Extra: LOOCV

Performing the same steps with Leave-One-Out Cross-Validation, the RMSE is 10.96285, compared to the original 10.86391. The difference here is caused by the different choice in  $k$ . Here it is 15, in the original cross-validation it is 13.

```
set.seed(100)
train.idx <- sample(nrow(df), 0.8*nrow(df))
train.data <- df[train.idx,]
test.data <- df[-train.idx,]

set.seed(101)
knn <- train(gain~air_time, data=train.data, method="knn",
             trControl=trainControl("LOOCV"),
             preProcess=c("center", "scale"),
             tuneLength=20)

plot(knn)

yhat <- predict(knn, test.data)
y <- test.data$gain
RMSE(yhat, y)
ggplot(data.frame(yhat, y), aes(y, yhat)) + geom_point() +
  geom_abline(aes(slope=1, intercept=0), col="red") +
  labs(title="kNN model performance", x="Observed", y="Predicted")
```

#### 4.7 Extra: random seeds

Random seeds can change your results greatly. Most times, the change is caused by the difference in the train and test set. In practice, don't change the seed to get a better result. Your model is what it is.

The model below achieves a test RMSE of 16.4329. Our original full model had a test RMSE of 10.86391.

```
set.seed(69420)
train.idx <- sample(nrow(df), 0.8*nrow(df))
train.data <- df[train.idx,]
test.data <- df[-train.idx,]

set.seed(42069)
knn <- train(gain~air_time, data=train.data, method="knn",
             trControl=trainControl("cv", number=6),
             preProcess=c("center", "scale"),
             tuneLength=20)

plot(knn)
```

```
yhat <- predict(knn, test.data)
y <- test.data$gain
RMSE(yhat, y)
ggplot(data.frame(yhat, y), aes(y, yhat)) + geom_point() +
  geom_abline(aes(slope=1, intercept=0), col="red") +
  labs(title="kNN model performance", x="Observed", y="Predicted")
```

#### 4.8 Extra: choice of tuneLength

In the training of the kNN model here, I set `tuneLength = 20`, an increase from the `tuneLength = 10` from the tutorial notes. I chose a higher `tuneLength` to test more values of  $k$ , so that I can be sure that the  $k$  I chose is the 'best'.

Past a certain point, (in this case,  $k = 30$ ), in general, the model's RMSE should get worse and worse - as  $k$  increases, each prediction made will get closer and closer to the mean of the training data. At the limit where  $k$  is the size of the dataset, every prediction made by the model will be the average of the dataset.

If there is truly any predictive power in your predictors, the 'global' mean will be a significantly worse predictor than a 'localised' mean, which is what the kNN model uses for its predictions.